Step 5 — Il training (LoRA), passo per passo

Obiettivo

Addestrare un adapter LoRA per ciascun modello sul dataset renderizzato (03) con la loss di 04, controllando durante il training apprendimento, calibrazione e oblio, e salvando checkpoint confrontabili.

Input e output

Input	pesi del modello (Winnow: `winnow-12b-bf16/model/`; JPT: `kirp/jpt-9b` revisione `b447cc7` nella cache HF), dataset renderizzato, configurazione
Output	`finetune/outputs/<esperimento>/`: adapter per checkpoint, `config.json`, log delle metriche, curve
Codice	`finetune/train.py` (**da scrivere**)

1. Che cos'è LoRA, in breve

Invece di aggiornare una matrice di pesi W (es. 4.096 × 4.096), LoRA aggiunge due matrici piccole: $W' = W + B \cdot A$, con A di forma r × 4.096 e B di forma 4.096 × r. Con r = 16, i parametri addestrati per quella matrice passano da ~16,8 milioni a ~131.000. W resta congelata.

Conseguenze pratiche:

- memoria: si conservano gradienti e stati dell'ottimizzatore solo per A e B;
- sicurezza: il modello di partenza è intatto, l'adapter si può togliere in qualsiasi momento;
- costo: un adapter pesa decine o centinaia di MB, non 20 GB.

2. Configurazione di partenza, e perché

Parametro	Valore	Perché
Moduli LoRA	tutti i proj di attention (q, k, v, o) e MLP (gate, up, down)	lo stesso schema usato per creare JPT; il compito tocca sia "dove guardare" (attention) sia "cosa concludere" (MLP)
Rank r / alpha	16 / 32	r = 16 è bastato per insegnare a JPT l'intera interfaccia di decisione; qui il compito è più stretto
Dropout LoRA	0,05	regolarizzazione leggera
Learning rate	2e-5, warmup 3%, decadimento cosine	più basso dei 5e-5 usati per JPT: stiamo continuando un modello già addestrato e vogliamo spostarlo il meno possibile
Epoche	1	i dati sono abbondanti e ripetitivi; la seconda epoca tende a sovra-adattare e peggiorare la calibrazione
Batch effettivo	32 esempi	stabile per LoRA; ottenuto con accumulo dei gradienti
Lunghezza massima	4.096 token (pilota), 8.192 (corsa piena)	vedi lunghezze in 02e
Precisione	bf16; gradient checkpointing	memoria
Ottimizzatore	AdamW, weight decay 0 sugli adapter	standard per LoRA
Loss	CE sulle lettere + λ·KL di replay (λ = 1)	04

Parametro	Valore	Perché
Checkpoint	ogni 10% dell'epoca	per scegliere il punto migliore sul dev
Seed	fisso e salvato (3 seed per la configurazione finale)	riproducibilità e stima della varianza

3. Hardware

Macchina	JPT-9B	Winnow-12B
Portatile RTX 5090, 24 GB	**QLoRA** (pesi base a 4 bit, ~5–6 GB), 4k token, batch fisico 1	non realistico
L40S 45 GB	LoRA bf16 (~18 GB di pesi), fino a 8k token	LoRA bf16 (~22 GB di pesi), 4–6k token
2× L40S	data parallel: ogni scheda ha una copia, i gradienti degli adapter sono piccoli e passano bene anche su PCIe	idem

Stima della memoria per Winnow su L40S: pesi 22 GB + adapter e ottimizzatore ~1 GB + attivazioni con gradient checkpointing a 4k token ~6–10 GB → circa 30–35 GB su 45.

Nota su QLoRA: i pesi base quantizzati a 4 bit cambiano leggermente il modello. Va bene per iterare su JPT, ma i numeri definitivi vanno da un training in bf16 (o almeno da una valutazione dei pesi fusi in bf16).

4. Tempi (stima da verificare)

Pilota `pilot_text` in S1: ~29.000 coppie + ~4.000 di replay ≈ 33.000 esempi × ~600 token ≈ **20M token**. A ~1.000 token/s per GPU: **~5–6 ore** su una L40S. In S3 circa 2,7× gli esempi. La velocità reale va misurata nei primi 100 passi e scritta nel log.

5. Strumenti

Strumento	Pro	Contro
transformers + PEFT con Trainer custom (consigliato)	pieno controllo sulla loss (sovrascrivendo `compute_loss`), poche dipendenze	va scritto il trainer
ms-swift	è il framework con cui è stato addestrato JPT; supporta Qwen3.5 (layer DeltaNet) e Gemma-4	loss custom da adattare
Unsloth, Axolotl, LLaMA-Factory	veloci da configurare per l'SFT classico (esperimento P3)	loss sulle lettere non prevista

Per JPT servono versioni di transformers che supportano i layer DeltaNet di Qwen3.5 (nell'ambiente `factcheck`: transformers 5.14.1, la stessa usata da vLLM).

6. Il ciclo di training, nel dettaglio

```
per ogni micro-batch:
    1. forward del modello con adapter → logit solo nelle posizioni answer_pos
    2. z = logit delle lettere / temperatura
    3. loss = CE (fact checking) + λ·KL (replay) [+ termini opzionali]
    4. backward; ogni 32 esempi: passo dell'ottimizzatore, aggiornamento del learning rate
ogni 10% dell'epoca:
    5. salvataggio dell'adapter
    6. valutazione rapida (vedi §7)
```

Pseudo-codice di `compute_loss`:

```
def compute_loss(model, batch):
    out = model(input_ids=batch["input_ids"], attention_mask=batch["attention_mask"])
    z = letter_logits(out.logits, batch["answer_pos"], batch["letter_ids"], TEMPERATURE)
    fc, rp = batch["kind"] == "factcheck", batch["kind"] == "replay"
    loss = decision_loss(z[fc], batch["target"][fc], "ce") if fc.any() else 0.0
    if rp.any():
        loss = loss + LAMBDA * decision_loss(z[rp], batch["target"][rp], "kl")
    return loss
```

Per risparmiare memoria conviene calcolare solo lo stato nascosto finale e applicare la testa del vocabolario solo alla posizione `answer_pos` (invece di ottenere i logit di tutte le posizioni).

7. Cosa controllare durante il training

A ogni checkpoint, su campioni piccoli e fissi (pochi minuti):

Metrica	Su cosa	Andamento atteso	Allarme
Loss di training (fact checking)	batch correnti	scende, poi si appiattisce	oscillazioni forti → learning rate troppo alto
BAcc e AUC	`dev_select` (fino a 600 righe per fonte)	salgono, oltre la baseline	una fonte scende mentre le altre salgono; sotto la baseline dopo il 30% dell'epoca
ECE	`dev_select`	stabile o in calo	sale molto → sovra-confidenza
KL di replay	split di validazione del replay	resta bassa (es. $< 0{,}05$)	cresce costantemente → oblio
Test "solo claim"	200 righe di `dev_select` con documento vuoto	P(supportato) media resta bassa	sale → il modello guarda solo il claim

Esempio di lettura: se dopo il 50% dell'epoca la BAcc su `dev_select` sale ma la KL di replay è passata da 0,01 a 0,15 e l'ECE da 0,06 a 0,12, il modello sta imparando il compito ma dimenticando il resto: aumentare λ o abbassare il learning rate.

8. Scelta del checkpoint

1. Si scartano i checkpoint con KL di replay oltre la soglia scelta o con calo sui benchmark di oblio > 2 punti.
2. Fra i rimanenti, si sceglie quello con la BAcc macro più alta su `dev_select`.
3. Si conferma una volta su `dev_confirm`.

LLM-AggreFact non entra nella scelta (né il test né il dev): il suo risultato resta zero-shot (Step 0).

Non si guarda mai il test per scegliere il checkpoint.

9. Problemi tipici e soluzioni

Sintomo	Causa probabile	Cosa fare
La loss non scende	posizione `answer_pos` sbagliata, id delle lettere sbagliati, target invertiti	ripetere i test del rendering (03, §7)
La BAcc peggiora subito	learning rate troppo alto, o formato diverso dal benchmark	lr 1e-5; confronto token per token
Out of memory	sequenze lunghe	gradient checkpointing, batch fisico 1, lunghezza massima più bassa, logit solo in `answer_pos`

Sintomo	Causa probabile	Cosa fare
Probabilità tutte vicine a 0 o 1	sovra-confidenza	Brier, β·KL, temperatura rifittata sul dev
Benchmark di oblio in calo	replay insufficiente	$\lambda = 2$, quota di replay al 20%, lr più basso
Ottimo su `dev_select`, nessun guadagno su test_ood o AggreFact (visto alla fine)	il mix non copre i domini o il tipo di errore dei test esterni	più varietà: riassunti (SummEdits, coppie generate da documenti lunghi), altri domini; da decidere per la versione successiva, senza ritoccare quella valutata

10. Riproducibilità

Per ogni esperimento salvare in `finetune/outputs/<esperimento>/config.json`: hash del mix e del dataset renderizzato, revisione dei pesi, tutti gli iperparametri, seed, versioni delle librerie, macchina, durata.

Criteri per passare allo step successivo

- Almeno un checkpoint che batte la baseline su `dev_select` (confermato su `dev_confirm`) senza violare i limiti di oblio e calibrazione.
- Log e configurazione salvati.